

CS 498 RM - MP4 Imitation Learning

Due on 11:59PM Wednesday, December 10, 2025

November 19, 2025

Q0 - Time Spent

Please record and report the number of hours you spend on completing this MP. This question will not be graded.

MP Description and Background

In this MP, you will solve a simple pick-and-place task using behavior cloning. You have already solved a similar task in MP2 by explicitly defining each action the robot should take. In this MP, we will try a different strategy to solve the pick-and-place task by letting the robot directly learn from expert (human) demonstrations. The robot should perform the set of actions shown in Figure 1 and remain stationary afterward. Your trained policies must achieve certain success rates at Checkpoint 1 and Checkpoint 2 to receive full credit. The red sphere and green line attached to the gripper are there to guide operators to better complete teleoperation. This MP is separated into Part 1 and Part 2, as detailed below.

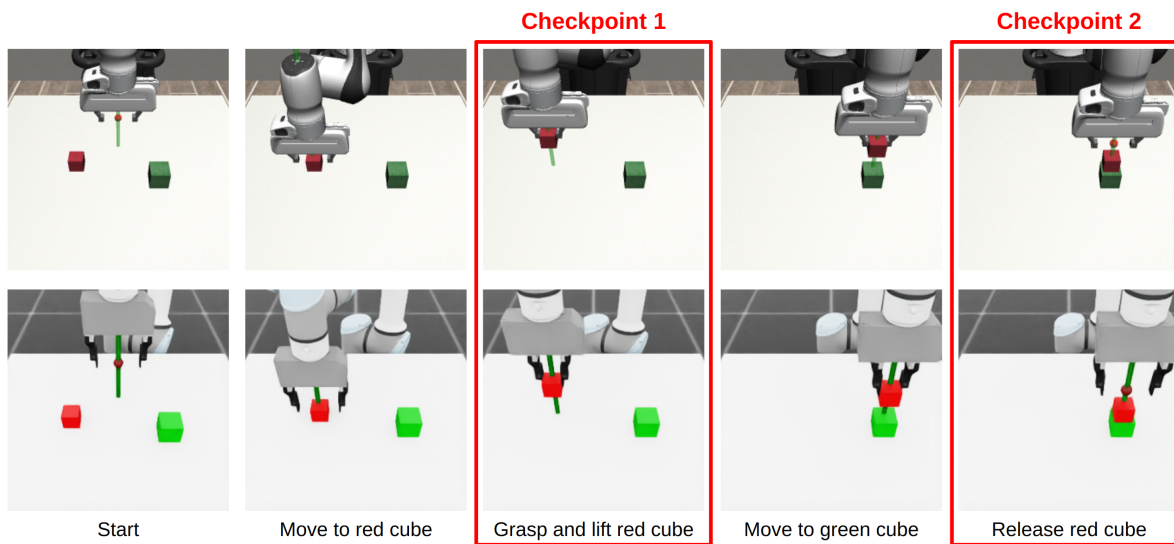


Figure 1: The two-cube pick-and-place task for this MP

The first step of behavior cloning would be to obtain a set of expert demonstrations. We will use keyboard teleoperation to collect human demonstration data, in which, given the observation at each time step, the human operator decides the appropriate action to take. The observation-action pairs extracted from demonstrations will be used to train a policy that maps the current observation to the next action.

- For Part 1, the observations will be the ground truth object state information (position and orientation, for example).

- For Part 2, the observations will be the RGB images obtained from a 3rd-person view camera.

At each time step, the robot can choose one action to perform among nine available actions: move in the $\pm x$ / $\pm y$ / $\pm z$ direction, open / close the gripper, or remain stationary. We will formulate policy learning as a classification problem and train a model such that, given the current observation, the model will select one of the nine available actions to take. The action at each time step is represented using one-hot encoding with format `[+x, -x, +y, -y, +z, -z, open_gripper, close_gripper, remain_stationary]`. As examples, `action = [0, 0, 0, 0, 1, 0, 0, 0, 0]` means the robot will move in the $+z$ direction at the current time step, and `action = [0, 0, 0, 0, 0, 0, 0, 0, 1]` means the robot will close its gripper at the current time step. Executing the predicted action will then alter the environment and produce the observations for the next time step, as shown in Figure 2.

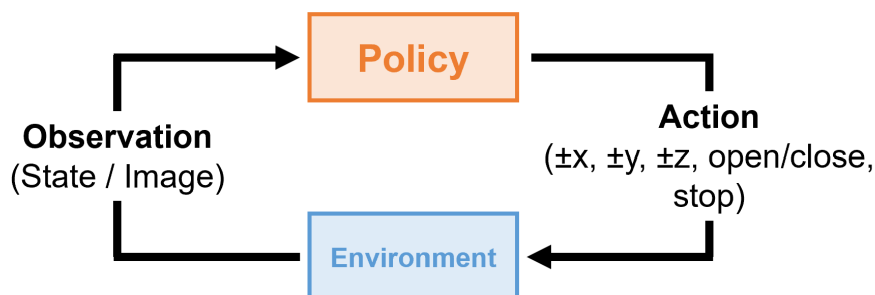


Figure 2: Interaction between the policy and the environment.

The provided starter code consists of six scripts (look for `TODO` segments in the scripts to add your code):

1. `task_envs.py`: This file contains the task environments for this MP. The position of the cubes will be randomized every time the environment is initialized. You are not allowed to modify this file.
2. `collect_demo.py`: This file provides functions for teleoperation and data collection. You can run it with `python3 collect_demo.py` for robosuite and `python3 collect_demo.py --enable_cameras` for IsaacLab. More details about this script are described below.
 - This script collects data for both Part 1 and Part 2.
 - When started, this script will record the actions you perform during teleoperation and store them under the directory specified in `dataset_dir`. Setting `dataset_dir = None` will let you play around with teleoperation without saving anything. The teleoperation key mappings will be printed in the terminal. You can change the key mappings if you wish.
 - The script collects a new data point if the gripper has changed its state (from open to closed or vice versa), or the robot has traveled sufficiently far from where the last data point was collected. The distance the robot needs to travel for a new data point to be collected is specified with the variable `pos_change_threshold`. It has a default value of 0.005m, but you can modify it if needed.
 - At any point during teleoperation, you have the option to start a new episode with or without saving the data collected during the current episode. The key mappings for these options are printed in the terminal at the start of each episode.
 - 10 additional data points with action = remain stationary will be automatically added to the end of every demonstration episode you collected.
 - For each data point, the script collects the following information:
 - (a) `cur_observation[:3]` – xyz position of the robot end-effector (for Part 1)
 - (b) `cur_observation[3]` – Robot gripper state (for Part 1)
 - (c) `cur_observation[4:7]` – xyz position of the red cube (for Part 1)
 - (d) `cur_observation[7:]` – xy position (z is irrelevant) of the green cube (for Part 1)

- (e) `cur_action` – The action taken at the current time step, using one-hot encoding (for Part 1 and Part 2)
- (f) Image from `robotview.camera` (for Part 2)

While the state information at each time step (`cur_observation` and `cur_action`) is saved into a combined `states.npz` file (keys: `state_observations` and `actions`) at the end of each teleoperation episode, the images are being saved under the `/images` folder while the teleoperation is still ongoing. If you select not to save the data from the current episode, the images will be deleted automatically. However, if you directly use `ctrl+c` to terminate the script, you would need to manually delete the images already saved.

- You can change how fast the robot moves by changing `temp_dist_target` in robosuite, and `max_join_change` and `temp_dist_target` in Isaac Lab.
3. `visualize_trajectories.py`: You will write your trajectory visualization code for Q1 and Q4 in here.
 4. `train_mlp.py`: This file is where you will train a state-based policy with the demonstrations you have collected with `collect_demo.py`. We have provided a basic PyTorch dataset definition, but you would need to define the model architecture, specify the hyperparameters, and complete the training loop yourself. You don't have to use the starter code we provided.
 5. `train_cnn.py`: This file is where you will train an image-based policy with the demonstrations you have collected with `collect_demo.py`. We have provided a basic CNN-based image encoder (you might want to further optimize its architecture, but what we provided should be a good start) to compress images into lower-dimensional representations. The provided starter code expects the input image to have range `[0, 255]` since it includes a `/255` operation that converts the image to range `[0, 1]`. You would need to complete the model architecture to map the output of the image encoder to robot actions, specify the hyperparameters, and complete the training loop yourself. You don't have to use the starter code we provided.
 6. `inference.py`: This file is where you will deploy the trained models in the test environment. You can run it with `python3 inference.py` for robosuite and `python3 inference.py --enable_cameras` for Isaac Lab. More details about this script are described below.
 - In the script, you would need to specify whether you are running the state-based policy ("`mlp`") or image-based policy ("`cnn`"), as well as the `model_path`.
 - You would need to update the model definition and input based on your model architecture. Look for the `TODO` sections in the script.
 - The cube positions will be randomized in the same way they are randomized in `collect_demo.py`. Please do not modify the environment/scene reset code.
 - For both robosuite and Isaac Lab, we have mapped a key to starting a new episode, and the mapping is printed in the terminal when the script starts. This should make it easier to screen record consecutive runs.
 - You can change how fast the robot moves by changing `temp_dist_target` in robosuite, and `max_join_change` and `temp_dist_target` in Isaac Lab.

In your Canvas submission, please include the following:

1. A typed report that answers all questions in this document. The questions to be answered are in **bold font**.
2. A zip file of the entire MP code folder, including the final models you trained, files we provided, and files you modified/created.
3. Videos demonstrating your trained policies, as detailed in Q3 and Q4.

Important Tips

GPU Usage

While Part 1 should be very doable on CPUs, you would need a GPU to train the policy for Part 2. You can upload your dataset to Google Drive and use Google Colab for GPU-based training. Once you obtain the final model `.pth` file, you can download it and run inference locally. Although the free version of Colab has usage limits, \$10 or 100 compute units is equivalent to 7–8 GPU training hours and should be enough for this MP. You would need to use Colab with your personal Google account if you intend to purchase compute units, since the Google account associated with your school email does not have the purchase option enabled.

Demonstration Collection

When collecting demonstrations, try to be consistent with the strategy you use and avoid making too many mistakes. Since we are formulating the policy as a classification problem, similar inputs (observations) should lead to similar outputs (actions). You should avoid taking drastically different actions when the observations are the same.

Model Training

1. You should use a small portion of your dataset as validation data to monitor the model performance during training. Most of the time, an 80/20 train/validation split would be good.
2. When selecting the optimizer for your model, there are many options aside from the basic **SGD** (Stochastic Gradient Descent). For example, **Adam** generally converges faster than **SGD**. You can read more about the available optimizers in PyTorch [here](#).
3. If the training loss is not going down, the model is not learning. You should adjust the learning rate and/or model architecture.
4. If the training loss is going down but the validation loss is going up, the model is overfitting. Some methods to mitigate overfitting include adding dropout layers, using weight decay, and implementing early stopping.

robosuite

1. Running this MP on FastX would be very slow. Therefore, please use the physical Linux EWS machines at Grainger if your local machine cannot run robosuite or cannot run this MP fast enough. Please refer to the “Running simulators on school computers” document on Canvas if needed.
2. When using EWS, please make sure the OpenCV window is at the top for the scripts to capture keyboard inputs.
3. When collecting demonstrations and running inference for Part 2, please monitor the image display closely to make sure the camera image is not corrupted. On EWS machines, running `export MUJOCO_GL="glx"` in the terminal once before you run the MP scripts should prevent any corruption from happening.
4. For all images (RGB and depth) obtained from robosuite, you would need to flip them in the x-axis. You can do so using functions such as `cv2.flip()`.

Isaac Lab

1. `cv2.imshow()` might not work in Isaac Lab. We do not recommend trying to fix it by modifying the OpenCV installation, as this might break the entire Isaac Lab setup. You can use `matplotlib` as an alternative for image visualization.
2. Please use `gripper_center` as the end-effector frame.

Part 1 - State-based Policy (50%)

In this part, you will use the ground truth object state information as observations. The state information used as observations in the start code is described in the `collect_demo.py` part of MP Description and Background. You can modify what state information you include in the observations if you wish.

Q1 - Data Collection

Use `collect_demo.py` to collect 10 demonstrations. **In your report, please attach a plot that visualizes your demonstration trajectories (positions of the robot end-effector) in 3D and clearly indicate the start and end of each trajectory.** You can use 3D plotting libraries such as matplotlib, Open3D, and vedo for visualization.

Q2 - State-based Policy Training

In `train_mlp.py`, define a Multi-Layer Perceptron (MLP) architecture to map the object state information to robot actions. **Please include your model architecture in your report.** Then, write the training loop to train an MLP-based policy using the 10 demonstrations collected in Q1. **Please attach a train vs. validation loss plot and clearly label each curve.** In our implementation, the MLP-based policy does not take more than a few minutes to train even on a CPU. After training, run the trained policy with `inference.py` and **describe its performance qualitatively (for example, can the robot successfully move to the red cube).**

Note: Since the demonstration dataset has a small size, the trained policy might perform poorly, and that is fine for Q1. However, the robot should at least do something reasonable, such as moving in the correct general direction. If your robot is just performing random or completely incorrect actions, you might be doing something wrong.

Q3 - Further Tuning

If the policy performance is not ideal, potential ways of fixing it include:

1. Finetune the model training hyperparameters and/or the model architecture. We have provided some tips in the Important Tips section.
2. Collect more data. As mentioned in Lecture 19, dataset aggregation (DAGger) is an efficient way to obtain additional demonstrations that are more aligned with what the policy encounters during rollout. In our implementation, we let the initial policy run, stop it when it noticeably deviates from the correct trajectory, and then collect new teleoperated demonstration data starting from where the policy was stopped. This ensures the new demonstrations collected cover some of the observations the policy would likely encounter during rollout. You might need to implement this in a new script by combining `collect_demo.py` and `inference.py`. You can also implement dataset aggregation in other ways.

Please try one or both methods described above and retrain a policy. **In the report, please briefly describe what you did to improve the policy performance, unless your policy from Q2 already performs well enough.** You will receive full credit if your trained policy achieves at least 80% success rate at Checkpoint 1 and 60% success rate at Checkpoint 2 across at least 10 **consecutive** runs. The criteria for Checkpoint 1 and Checkpoint 2 to be considered successful are as follows:

- Checkpoint 1: The robot grasps onto and lifts the red cube off the table without dropping the cube.
- Checkpoint 2: After placing the red cube onto the green cube, the stack should remain stable for at least two seconds. Furthermore, the robot should not move after it finishes making the stack.

In your submission, please include a continuous screen recording of at least 10 consecutive runs. Please label the trial number for each run. In your report, please include a table detailing whether each run in the recording succeeded at Checkpoints 1 and 2. As mentioned in the `inference.py` part of MP Description and Background, you can start a new run by pressing a key while `inference.py` is running. **What are the common ways your trained policy fails (if applicable)?**

Part 2 - Image-based Policy (50%)

The ground truth object state information would be extremely hard to obtain (often requires attaching AprilTags or motion capture markers to the objects) if we work with real robots. Instead, roboticists have been trying to develop methods that are capable of predicting robot actions directly from the images the robot perceives. In Part 2, you will train an image-based policy using images captured by the `robotview.camera`. The general rules for this part are as follows:

- You are not allowed to use the ground truth position of the cubes as part of the policy input.
- You can modify the images with `cv2.flip()`, `cv2.cvtColor()`, and `cv2.resize()` if needed. Other operations, such as color thresholding, are not allowed.
- You can use predefined and pretrained models from `torchvision` if you wish.

Q4 - Image-based Policy Training and Evaluation

For this question, you will define, train, tune, and evaluate an image-based policy to complete the pick-and-place task. First, use `collect_demo.py` to collect additional demonstrations. Then, in `train_cnn.py`, define a model architecture to map the `robotview.camera` images to robot actions and complete the training loop. More details on `train_cnn.py` can be found in MP Description and Background.

You will receive full credit if your trained policy achieves a 60% success rate at Checkpoint 1 and a 30% success rate at Checkpoint 2 across at least 10 **consecutive** runs. Please still complete this question even if your policy does not meet the required success rate, as we will give partial credit. On the other hand, achieving a higher success rate will grant you up to 20% extra credit. For your final trained policy, **please report the following things:**

- Detailed model architecture, including the policy input.
- A plot that visualizes all your demonstration trajectories used for training.
- A plot that visualizes your final policy's train vs. validation loss.
- Any training and data collection strategies you incorporated to improve the policy performance.

In your submission, please include a continuous screen recording of at least 10 consecutive runs. Please label the trial number for each run. In your report, please include a table detailing whether each run in the recording succeeded at Checkpoints 1 and 2. As mentioned in the `inference.py` part of MP Description and Background, you can start a new run by pressing a key while `inference.py` is running. **What are the common ways your trained policy fails (if applicable)?**

Additional tips for this question are as follows:

- We used the image encoder provided in the starter code with very minor modifications. Even without further modifications, the provided CNN-based image encoder should give you a policy that at least does something reasonable.

- We were able to train a decent policy with around 4500 data points (completing one demonstration with the default `pos_change_threshold` value produces 100 – 120 data points), and the final model took about 20 minutes to train.
- Incorporating some form of dataset aggregation might be helpful.
- Incorporating history information might be helpful. This history information could be images or actions from previous time steps.

Q5 - Reflection

You have now solved a cube pick-and-place task in both MP2 and this MP using completely different methods. **In your report, share your thoughts on all three prompts below.** There are no wrong answers; you will receive full credit as long as you justify your responses.

1. Which method do you prefer and why?
2. Although the task in MP2 is not exactly the same as this MP, do you think the method you developed for MP2 (with proper modifications to suit the task in this MP) would have outperformed the policies you trained in this MP?
3. In your opinion, what are the advantages of using imitation learning to solve robotic manipulation tasks (if any)?

Extra Credit

Completing the task(s) described below will grant you extra credit up to 20% of the total available points from this MP. If you attempted the extra credit task but failed to finish it, feel free to describe and showcase what you have done, and we might give you partial extra credit based on your results. **We might or might not have verified whether it is actually possible to complete the extra credit task(s), and we cannot guarantee the amount of extra credit we offer is proportional to the time required to complete the extra credit task(s). Please invest your time accordingly.**

For this MP, you can earn up to 20% extra credit by achieving higher success rates at Checkpoint 1 and Checkpoint 2 than the required 60% and 30% for the image-based policy. The amount of extra credit awarded depends on your trained policy success rate.